David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

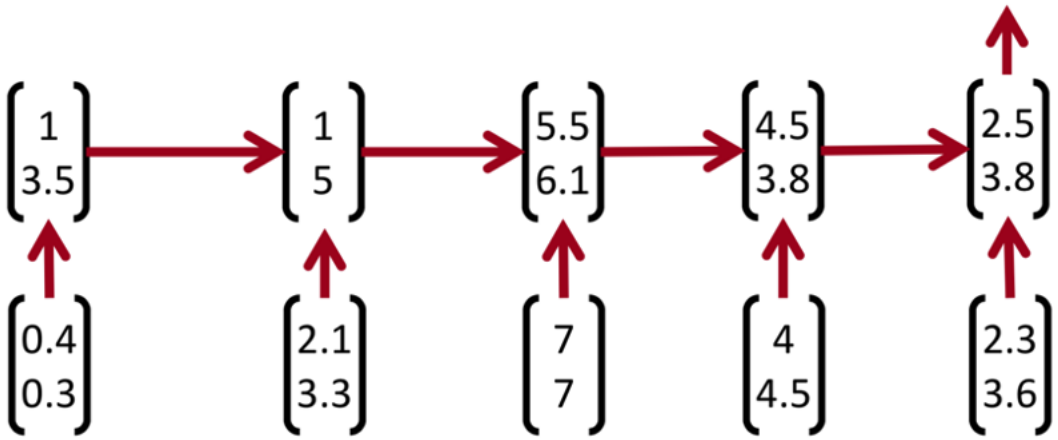
https://cs.ucf.edu/~mohaisen/

CNN-Based Models for Text Representation

① RNN Limitations
② CNN Motivation
③ Phrase-Level Convolution
④ Pooling Mechanisms
⑤ CNN Architecture
⑥ Sentence Classification

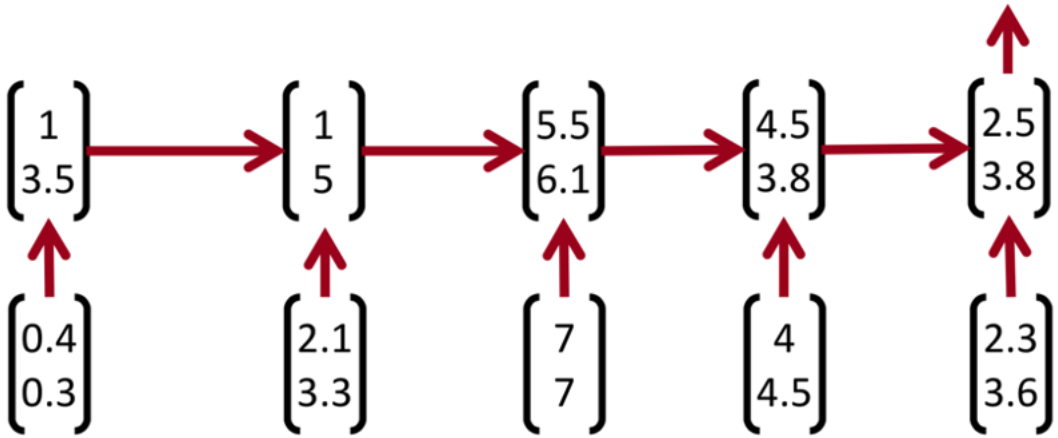
Limitations of RNNs in Capturing Phrase

- **Prefix Dependency Issue**
  - RNNs process sequences **one step at a time**, updating the hidden at each step.
  - This sequential dependency means that **phrases without a strong prefix context** are often not well captured.
  - Example: **Input:** "The book that I borrowed was fascinating."
    - RNN may struggle to connect "**book**" with "**was fascinating**" if intermediate words dilute the meaning.



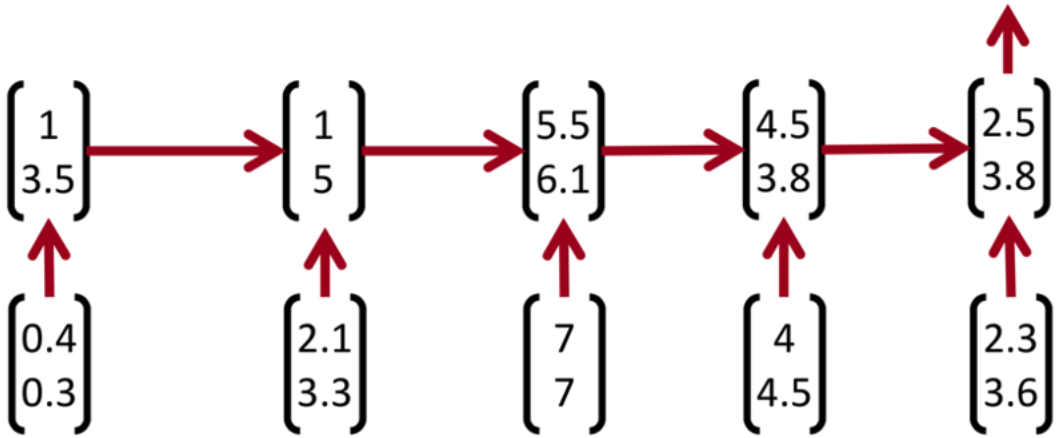
Limitations of RNNs in Capturing Phrase

- **Overweighting of Recent Tokens**
  - The final hidden state **disproportionately** represents the last few tokens.
  - Early tokens in the sequence **fade in memory**, leading to an implicit **recency bias**.
  - Example:
    - **Input:** "The performance was excellent, despite a slow start."
    - RNN may **overemphasize** "slow start" in its final representation.



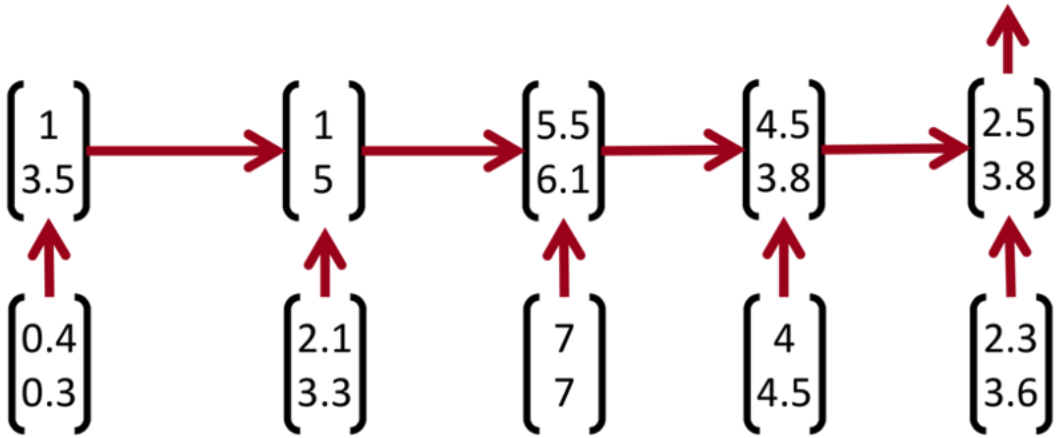
Limitations of RNNs in Capturing Phrase

- **Softmax at the Last Step**
  - The **softmax activation** at the output layer is computed only **at the last timestep**.
  - This means that all previous words **only contribute indirectly** through the hidden state.
  - The final output can **misrepresent** sentence meaning if crucial information appeared earlier.



Limitations of RNNs in Capturing Phrase

- **Implications**
  - RNNs struggle with **long-range dependencies**.
  - Alternatives like **Transformer models** address this by using **self-attention**, which considers the entire sequence at once.
  - Other alternative? CNN



CNN for Sequence Modeling: Limitations

- **Limited Long-Range Dependencies**
  - CNNs rely on **local receptive fields**, so capturing long-term dependencies requires **deep stacks of convolutions** or **dilated convolutions**.
- **Lack of Adaptive Attention**
  - Unlike Transformers, CNNs **don't dynamically weight different parts of the input**—they apply **fixed filters** regardless of the context.
- **Increased Depth for Large Contexts**
  - To capture long dependencies, CNN models must increase their depth, leading to vanishing gradient issues or large parameter overhead.

CNN for Sequence Modeling: Merits

- CNNs are useful for tasks where:
  - **Phrase-level features** are important (e.g., sentiment analysis, text classification).
  - **Efficiency** is a concern (since CNNs are more parallelizable than RNNs).
  - **Fixed-length context windows** suffice.
- **However, for tasks requiring long-range dependencies, Transformers remain the best alternative to RNNs.**

The Main Idea of CNNs for NLP

- **Convolutional Neural Networks (CNNs) for Text Processing**
  - **Key Concept:** Instead of processing text sequentially (like RNNs), CNNs **compute vector representations** for every **possible subsequence** of a certain length.
    - This enables **parallel feature extraction** at different positions in the input.

"tentative deal reached to keep government open"

CNN

CNN extracts the following **n-gram feature vectors** (for window size = 3):

1. tentative deal reached
2. deal reached to
3. reached to keep
4. to keep government
5. keep government open

✅ **Captures phrase-level features** regardless of grammatical correctness
✅ **Works in parallel**, unlike RNNs
❌ **Not linguistically or cognitively plausible** (fixed window ignores sentence structure)
✅ **Feature extraction is position-invariant** (doesn't matter where a phrase appears)

How CNNs Aggregate Subsequence Features?

- **Step 1: Convolution (Feature Extraction)**
  - Apply **filters (kernels)** of fixed size over the text to capture **local n-gram patterns**.
  - Each filter produces a **feature map**, highlighting specific **semantic/structural patterns**.
- **Step 2: Pooling (Feature Selection & Reduction)**
  - **Max Pooling:** Selects the most important feature from each filter's output (e.g., strongest phrase).
  - **Average Pooling:** Averages feature activations across all positions.
- **Example:**
  - Given the feature vectors from "**tentative deal reached to keep government open**", **max pooling** selects the **strongest signals** from each filter.
  - This results in a **fixed-length representation** of the sentence, independent of input size.

✅ Reduces dimensionality while preserving **salient features**
✅ Improves **efficiency** compared to RNNs
❌ Doesn't capture long-term dependencies as well as attention mechanisms in Transformers

CNN Basics: 1D Discrete Convolution

- The **1D convolution** of a sequence x with a filter (or kernel) w is :

$$(y * x)[t] = \sum_{i=0}^{k-1} w[i] \cdot x[t-i]$$

- Where:
  - $x[t]$: Input sequence (e.g., a sentence as a sequence of word embeddings)
  - $w[i]$: Convolution filter of size k (kernel size)
  - $y[t]$: Output feature map
  - k: Size of the convolution filter (window size)
  - $*$: Convolution operation

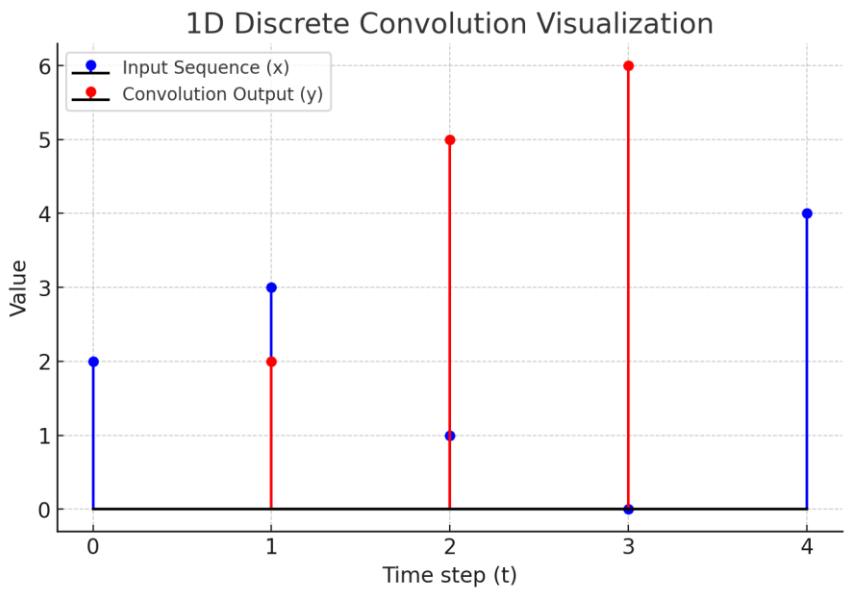
For position t=2:

x=[2,3,1,0,4] (input sequence)

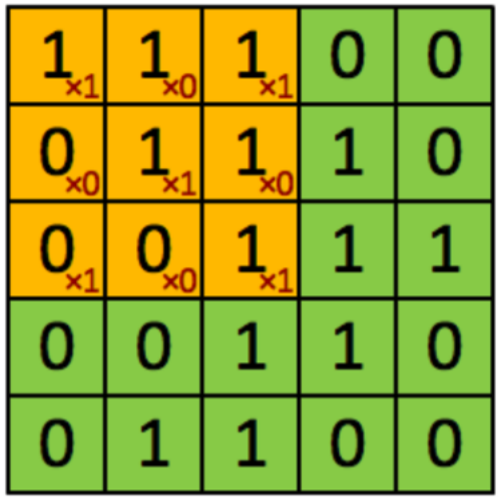
w=[1,−1,2] (filter of size k=3)

y[2]=(1·x[2])+(−1·x[1])+(2·x[0])

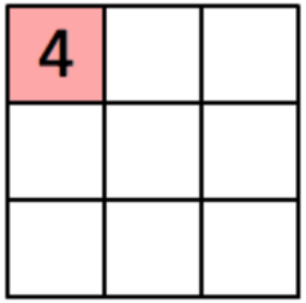
=(1·1)+(−1·3)+(2·2)=1−3+4=2

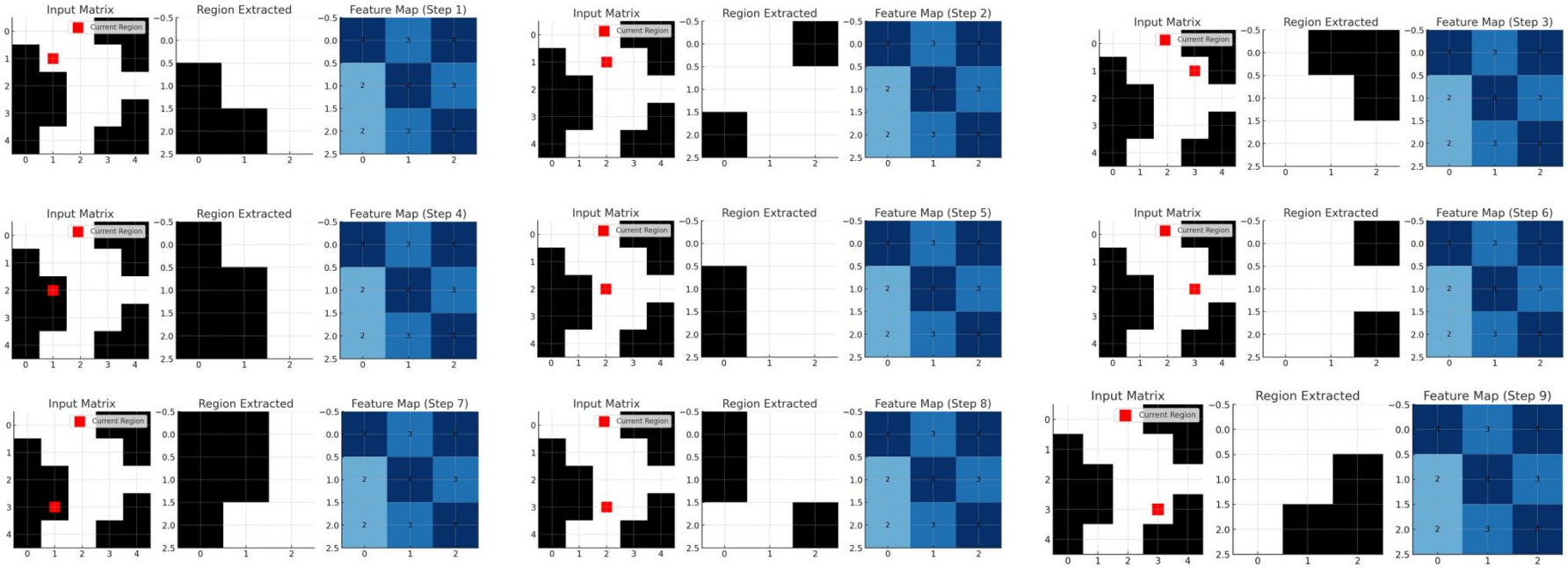


CNN Basics: 1D Discrete Convolution



Given a 2d matrix of [1 1 1 0 0, 0 1 1 1 0, 0 0 1 1 1, 0 0 1 1 0, 0 1 1 0 0] and a filter of [1 0 1, 0 1 0, 1 0 1] show step by step calculation of the feature map.





Discrete Convolution for Text

tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3

Apply a **filter** (or **kernel**) of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3

0.2*3 + 0.1 * 1 - 0.3* 2 - 0.4*3 - 0.5*1 + 0.2* 2 -0.3*1 +0.1*3 - 0.1*1 -0.3*1 +0.2*1 +0.4*1 = -1

Discrete Convolution for Text: Padding

Ø	**0.0**	**0.0**	**0.0**	**0.0**
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	**0.0**	**0.0**	**0.0**	**0.0**

Apply a **filter** (or **kernel**) of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

11

Ø,t,d	−0.6
t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3
g,o,Ø	−0.5

Discrete Convolution for Text: Channels

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Could also use (zero) padding = 2

Also called “wide convolution”

Discrete Convolution for Text: Max Pooling

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1

max p	0.3	1.6	1.4

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Discrete Convolution for Text: Averaging Over Time

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1

ave p	−0.87	0.26	0.53

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Discrete Convolution for Text: Stride = 2

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
d,r,t	−0.5	−0.1	0.8
t,k,g	−0.2	0.1	1.2
g,o,Ø	−0.5	−0.9	0.1

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Local Max Pooling with Stride

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1
Ø	−Inf	−Inf	−Inf

Ø,t,d,r	−0.6	1.6	1.4
d,r,t,k	−0.5	0.3	0.8
t,k,g,o	0.3	0.6	1.2
g,o,Ø,Ø	−0.5	−0.9	0.1

Conv1d, k-max pool; k=2

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1

2-max p	−0.2	1.6	1.4
	0.3	0.6	1.2

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

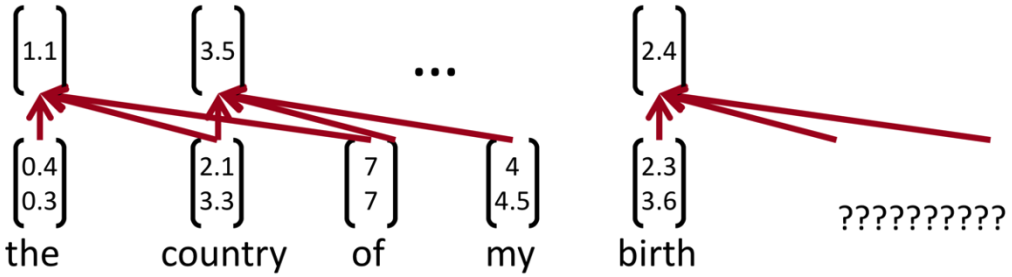
CNNs for Sentence Classification (Kim, 2014)

- **Yoon Kim (2014)** introduced **Convolutional Neural Networks (CNNs)** for **sentence classification**.
  - Inspired by a **2011 variant** of CNNs by **Collobert & Weston**.
  - CNNs effectively extract **key n-gram features** for **text classification** tasks.
- **Application to Sentiment Analysis**
  - CNNs can classify sentences as **positive** or **negative** in sentiment.
  - Unlike traditional methods, CNNs automatically learn **important feature representations**.
- **Other Applications of CNNs in NLP**
  - **Subjective vs. Objective Classification:**
    - Determine if a sentence expresses **opinion** or **factual information**.

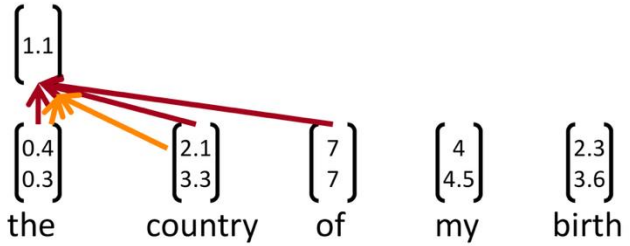
CNNs for Sentence Classification (Kim, 2014)

- Other Applications of CNNs in NLP
  - **Question Classification:** Categorize questions based on type:
    - **Person:** "Who is the CEO of OpenAI?"
    - **Location:** "Where is UCF located?"
    - **Number:** "How many states are in the US?"

✅ Captures phrase-level meaning efficiently
✅ Works in parallel, unlike RNNs
✅ Useful for short text classification tasks



- A simple use of one convolutional layer and **pooling**
- Word vectors: $\mathbf{x}_i \in \mathbb{R}^k$
- Sentence: $\mathbf{x}_{1:n} = \mathbf{x}_1 \oplus x_2 \oplus \cdots \oplus \mathbf{x}_n$ (vectors concatenated)
- Concatenation of words in range: $\mathbf{x}_{i:i+j}$ (symmetric more common)
- Convolutional filter: $\mathbf{w} \in \mathbb{R}^{hk}$ (over window of *h* words)
- Note, filter is a vector!
- Filter could be of size 2, 3, or 4:



CNNs for Sentence Classification (Kim, 2014)

- Pooling: max-over-time pooling layer
- Idea: capture most important activation (max over time)
- From feature map
- Pool single number:
- Use multiple filters weigh
- Useful to have different wi
- Because max pooling length **c** is irrelevant

$$\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}$$

$$\hat{c} = \max\{\mathbf{c}\}$$

$$\hat{c} = \max\{\mathbf{c}\},$$

$$\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}$$

CNN-Derived Features for Classification

- **Feature Extraction with CNN**
  - **Convolution** applies filters to extract **n-gram features** from text.
  - **Max pooling** selects the most important features, creating a **fixed-size feature map**.

- **Flattening and Fully Connected Layers**
  - The **pooled features** are **flattened** into a **1D feature vector**.
  - Passed to **fully connected layers (MLP/FFNN)** to learn **higher-level representations**.

- **Classification with Softmax/Sigmoid**
  - **Softmax activation:** Used for **multi-class classification** (e.g., question types).
  - **Sigmoid activation:** Used for **binary classification** (e.g., positive vs. negative sentiment).

✅ CNN extracts key features → Fully connected layers learn patterns → Softmax/Sigmoid makes final prediction.